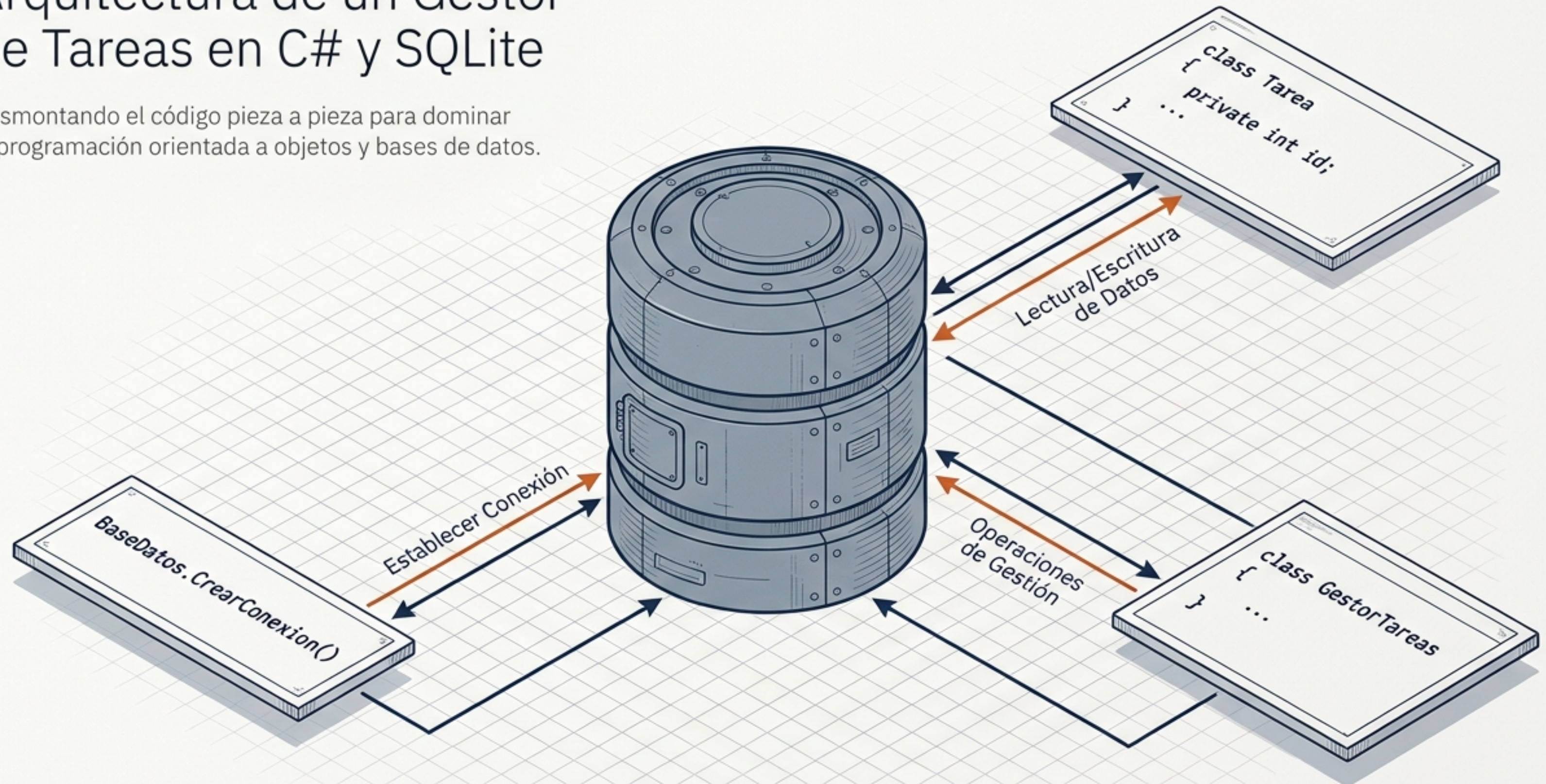
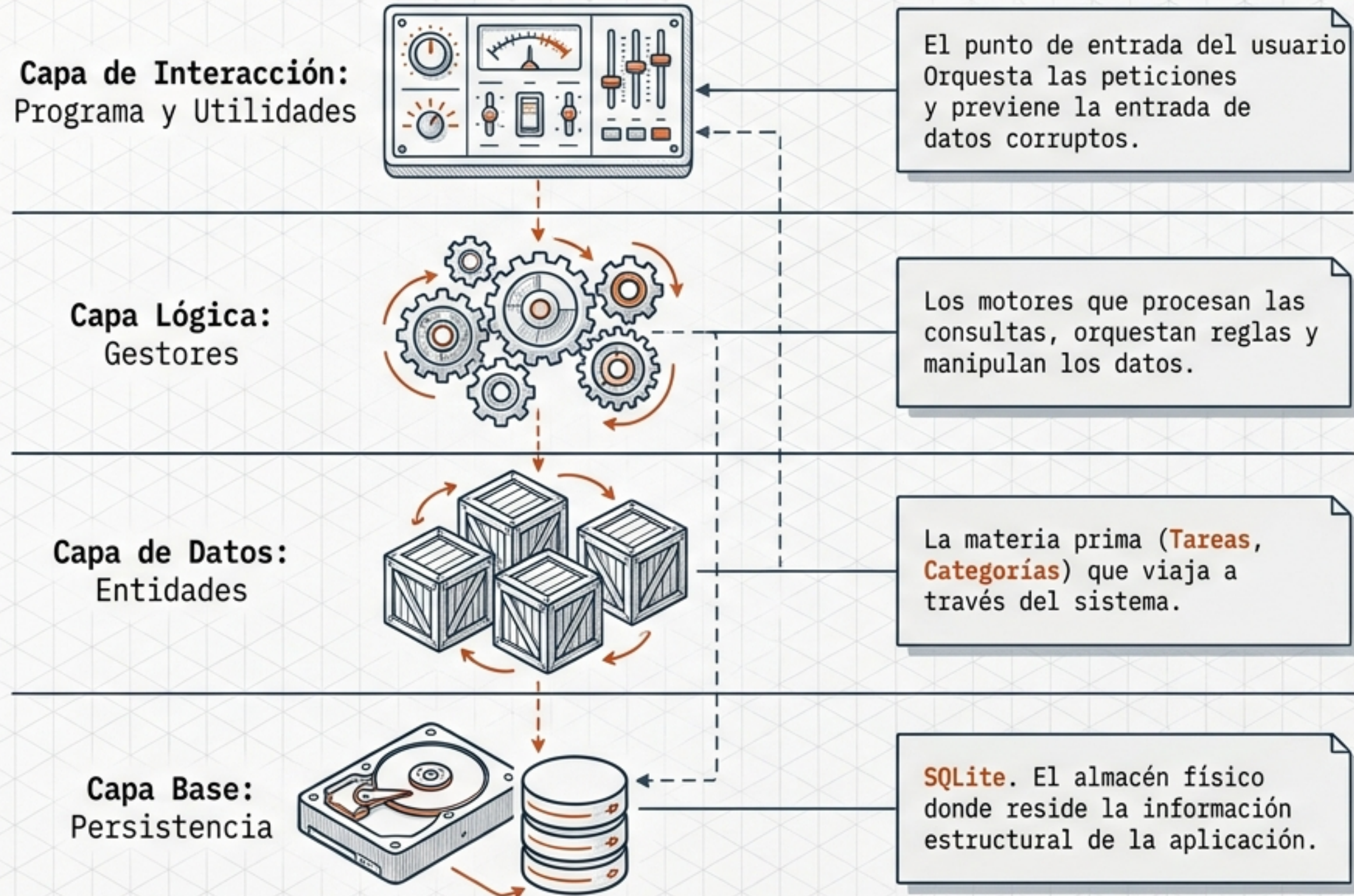


Arquitectura de un Gestor de Tareas en C# y SQLite

Desmontando el código pieza a pieza para dominar la programación orientada a objetos y bases de datos.



El Plano General: Macro-Arquitectura del Sistema



La **arquitectura modular** previene el "**código espagueti**" asignando una única responsabilidad a cada capa, facilitando el **aislamiento de errores**.

El Cimiento: Estableciendo la Conexión

Archivo: BaseDatos.cs

```
using Microsoft.Data.Sqlite;  
class BaseDatos {  
    private const string CadenaConexion = "Data Source=lista_tareas.db";  
}
```

Inmutable

La ruta a la base de datos es constante; nunca cambia durante la ejecución del programa.

Almacenamiento Local

El archivo físico local donde el motor SQLite escribirá y leerá todos los registros.

El Patrón Centralizado

- Evita la repetición de código.
- Si el archivo de base de datos cambia de nombre o ubicación, solo modificamos una línea en todo el proyecto.

Anatomía de una Entidad (I): La Caja de Datos

Archivo: Tarea.cs

```
class Tarea {  
    private int id;  
    private string titulo;  
    private string descripcion;  
    private bool completada;  
    private Categoria categoria;  
}
```

Tabla SQLite: Tareas

ID (INTEGER)
Titulo (TEXT)
Descripcion (TEXT)
Completada (INTEGER/BOOLEAN)
Categoria_ID (INTEGER)

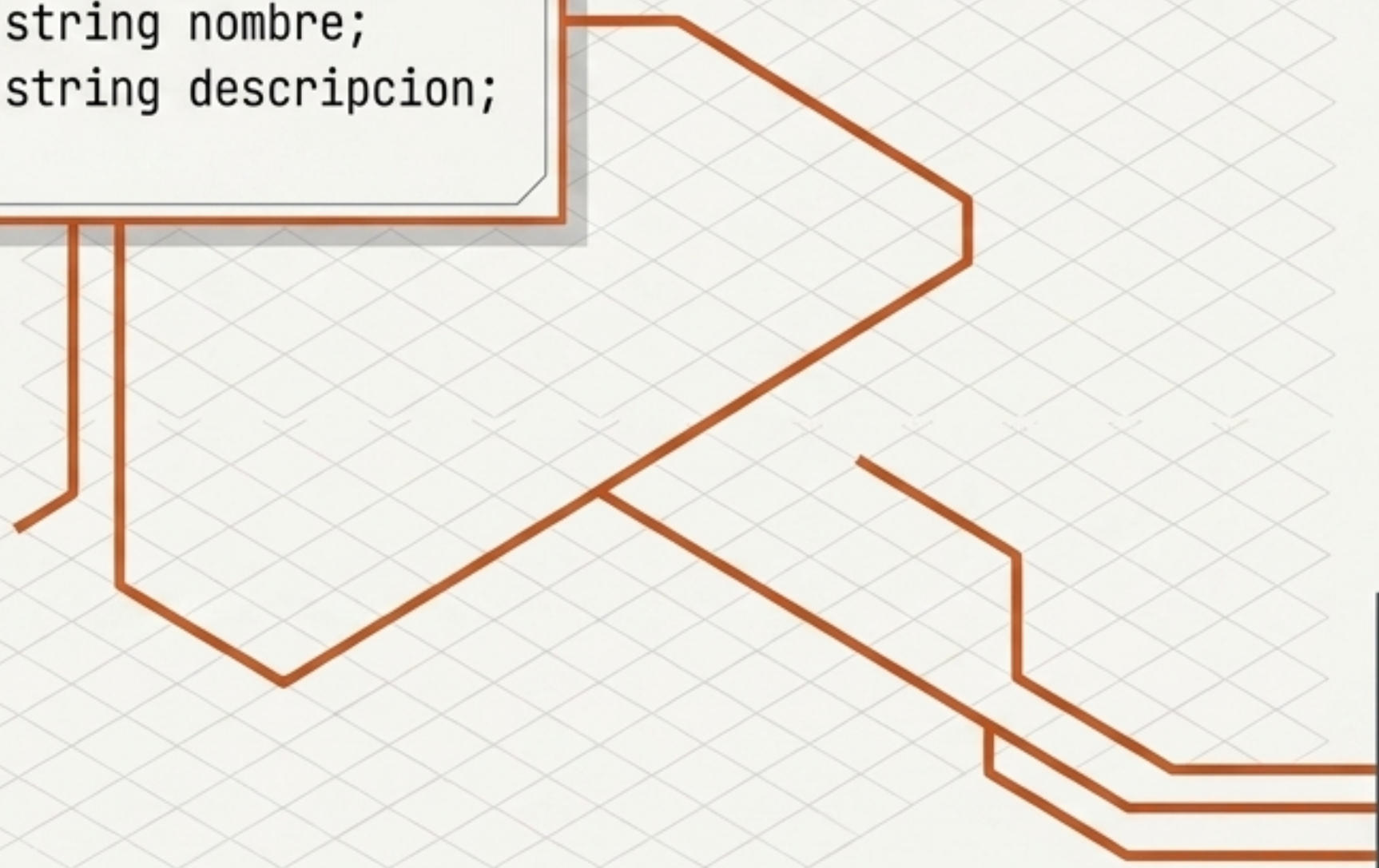
Mapeo Objeto-Relacional

El tipado estático de C# (int, string, bool) impone estructura y seguridad, actuando como un molde perfecto para los registros crudos de la tabla SQLite.

Anatomía de una Entidad (II): Relaciones

Archivo: Categoria.cs

```
class Categoria {  
    private int id;  
    private string nombre;  
    private string descripcion;  
}
```



Composición de Objetos

En el mundo relacional (SQLite), una Tarea almacena solo un número identificador de su categoría. Sin embargo, en el paradigma de Orientación a Objetos (C#), la Tarea Tarea contiene una referencia al objeto Categoria completo.

Esto refleja cómo interactúan los conceptos en el mundo real, permitiendo acceder a `tarea.categoria.nombre` directamente.

```
class Tarea {  
    // ... propiedades  
    private Categoria categoria;  
}
```


Matriz Diagnóstica: Entidades vs. Gestores

	Entidades (Tarea, Categoría) 	Gestores (GestorTareas) 
Propósito	Almacenar estado y representar los datos (El Qué).	Ejecutar lógica operacional y manipular datos (El Cómo).
Naturaleza del Estado	Mutable : Los valores de sus propiedades cambian constantemente (ej. tarea completada).	Stateless : No guardan información del usuario, solo almacenan dependencias funcionales.
Relación con SQLite	Representan pasivamente una fila dentro de una tabla de la base de datos.	Activamente escriben, ejecutan y leen comandos SQL (Queries) hacia la base de datos.

El Motor de Operaciones

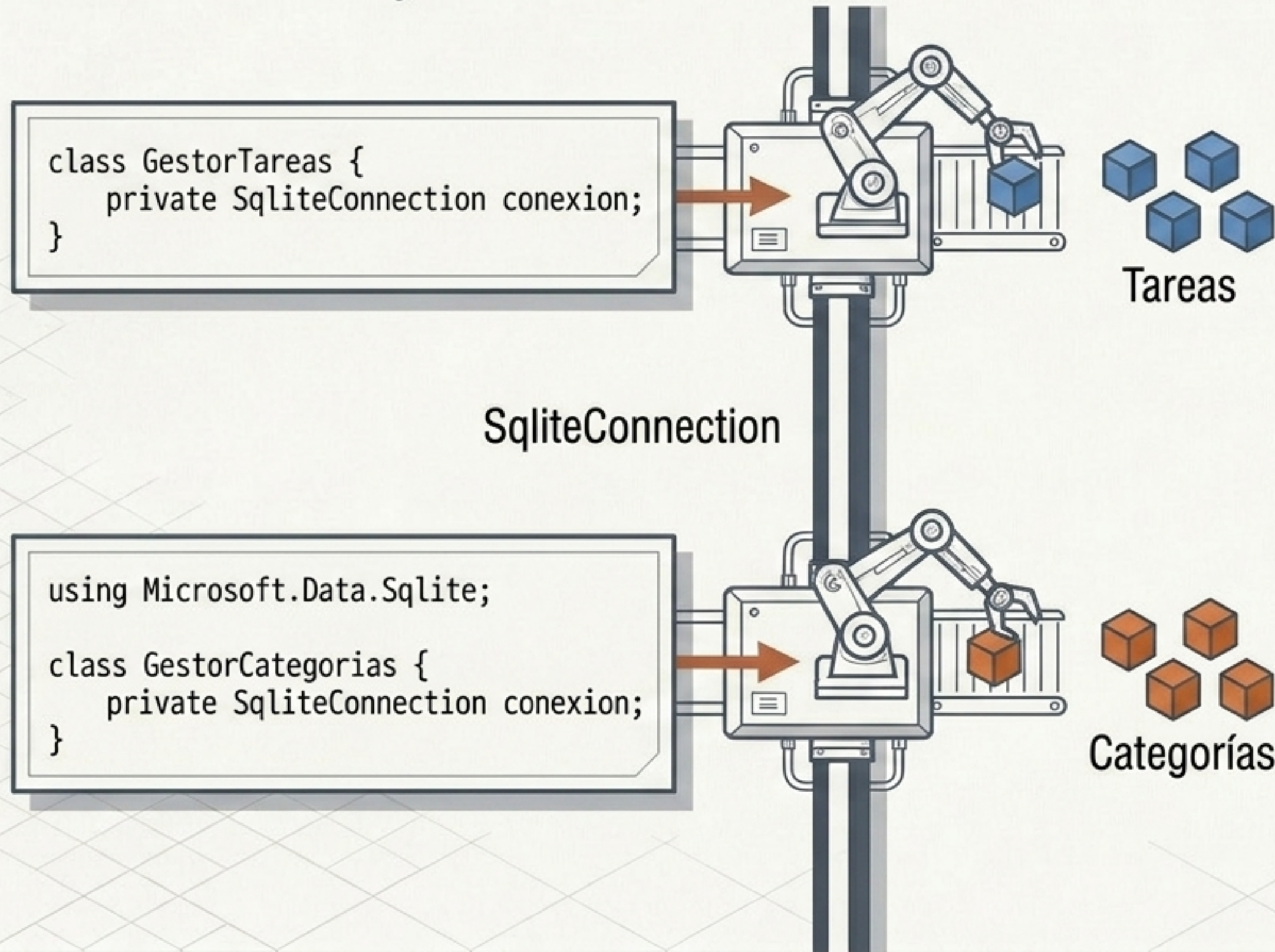
Archivo: GestorTareas.cs



Diseño por Inyección: El Gestor almacena una dependencia (la conexión SQLite). No la fabrica por sí mismo, sino que la recibe del exterior para utilizarla. Esto permite que el sistema de conexión centralizado opere sin fricciones.

Separación de Responsabilidades

Archivo: GestorCategorias.cs



Principio SoC (Separation of Concerns)

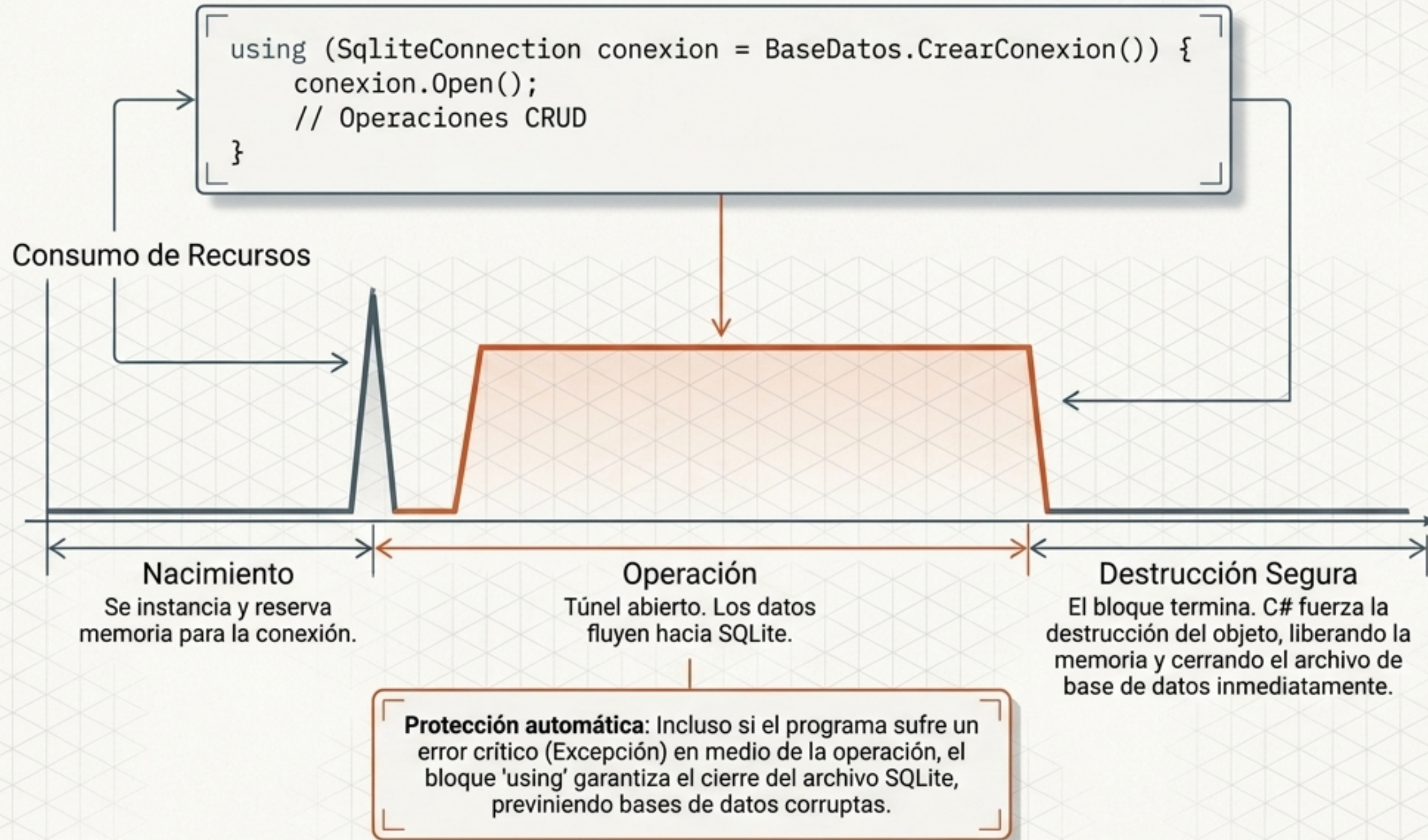
¿Por qué no poner todo en un archivo único de 'SuperGestor'?

Separar los gestores asegura que si hay un error en la lógica de las Categorías, el sistema de Tareas permanezca intacto.

Código más pequeño equivale a código testeable y escalable.

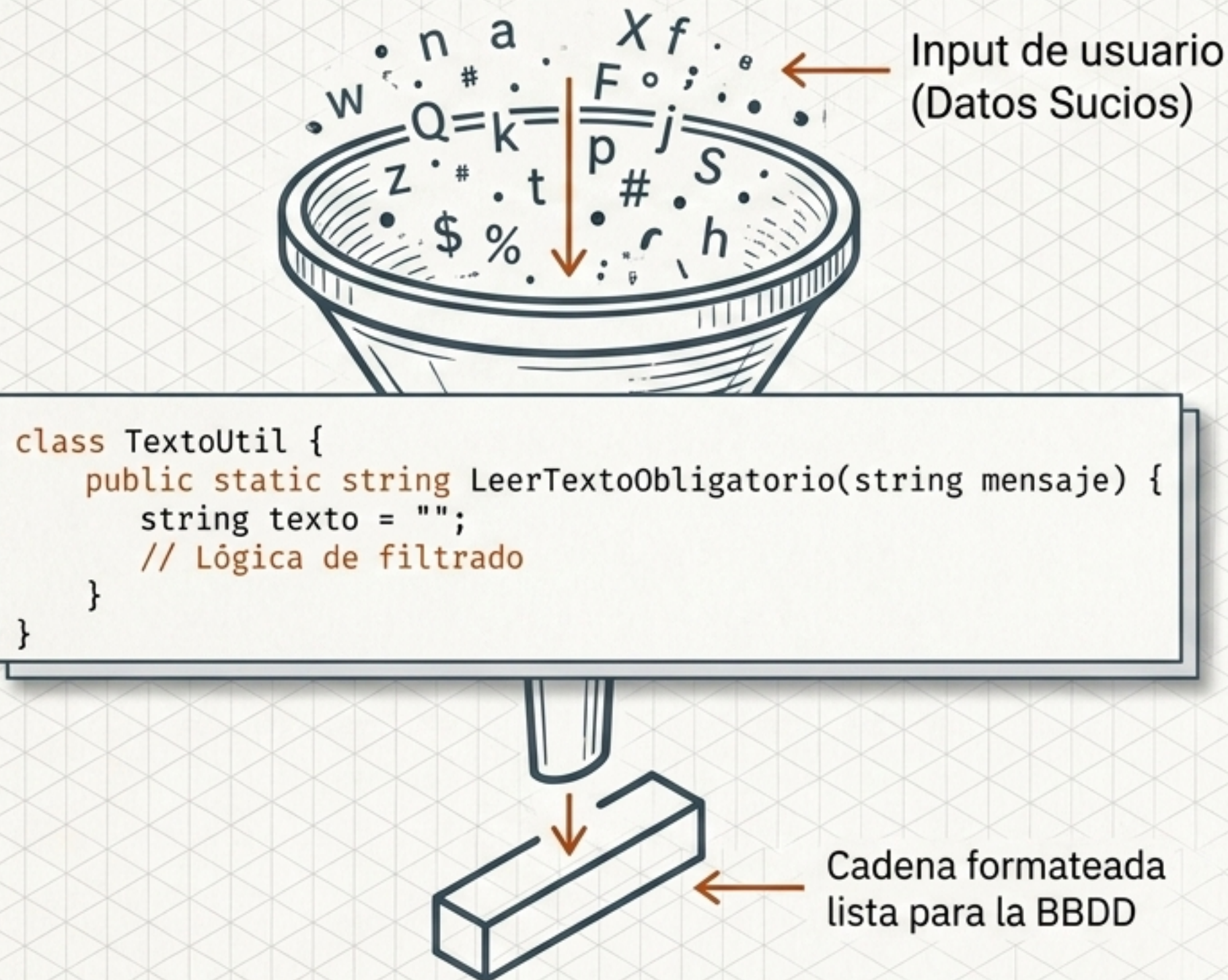
El Ciclo de Vida Seguro: Gestión de Memoria

La sintaxis 'using' y la interfaz IDisposable



El Filtro de Entrada: Control de Calidad

Archivo: TextoUtil.cs



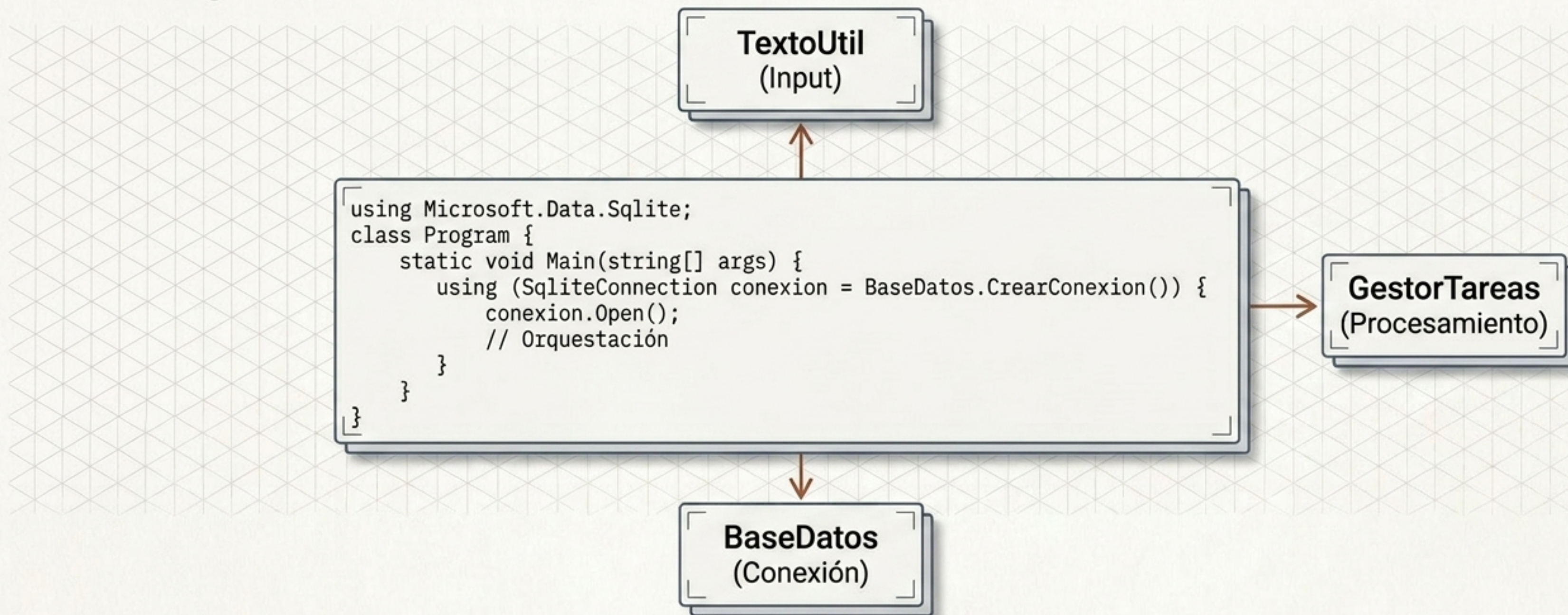
Metodología "Static"

¿Por qué esta clase y su método son **estáticos** (static)? Porque es una **herramienta de uso universal** en toda la aplicación.

Limpiar un texto **no requiere instanciar un objeto** con un estado único; es una pura **transformación funcional** matemática de entrada y salida.

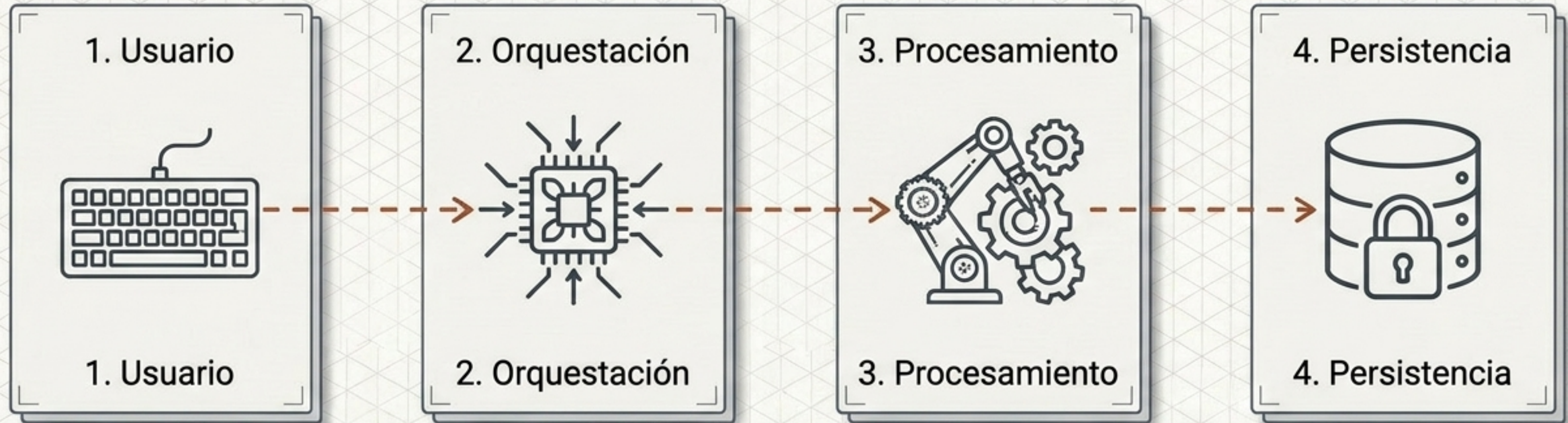
El Director de Orquesta

Archivo: Program.cs



Delegación pura. El método `Main` no inserta datos ni limpia textos manualmente. Su única responsabilidad estructural es iniciar la conexión, pedir datos usando las utilidades, y transferirlos a los gestores. Es el guion que coordina a los actores del sistema.

Flujo de Ejecución: El Ciclo Completo del Dato



El usuario teclea la nueva tarea.
"TextUtil" captura y purifica la entrada de la consola.

"Program" inicia el flujo de trabajo y recibe el texto de forma segura.

"GestorTareas" toma los datos, los empaqueta en parámetros y formula la consulta SQL.

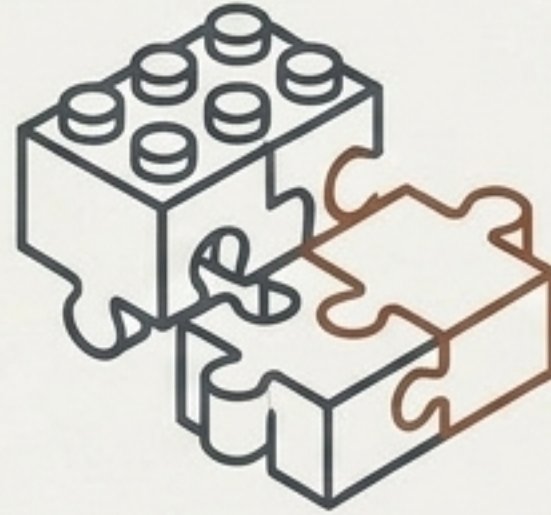
A través de "SqliteConnection", la información cruza la red local y queda sellada físicamente en el disco duro.

Principios Arquitectónicos Aplicados



Encapsulamiento

Uso riguroso de modificadores **'private'** en Entidades y Gestores. El estado interno y las conexiones a la base de datos están protegidos contra interferencias externas no controladas.



Reusabilidad y Modularidad

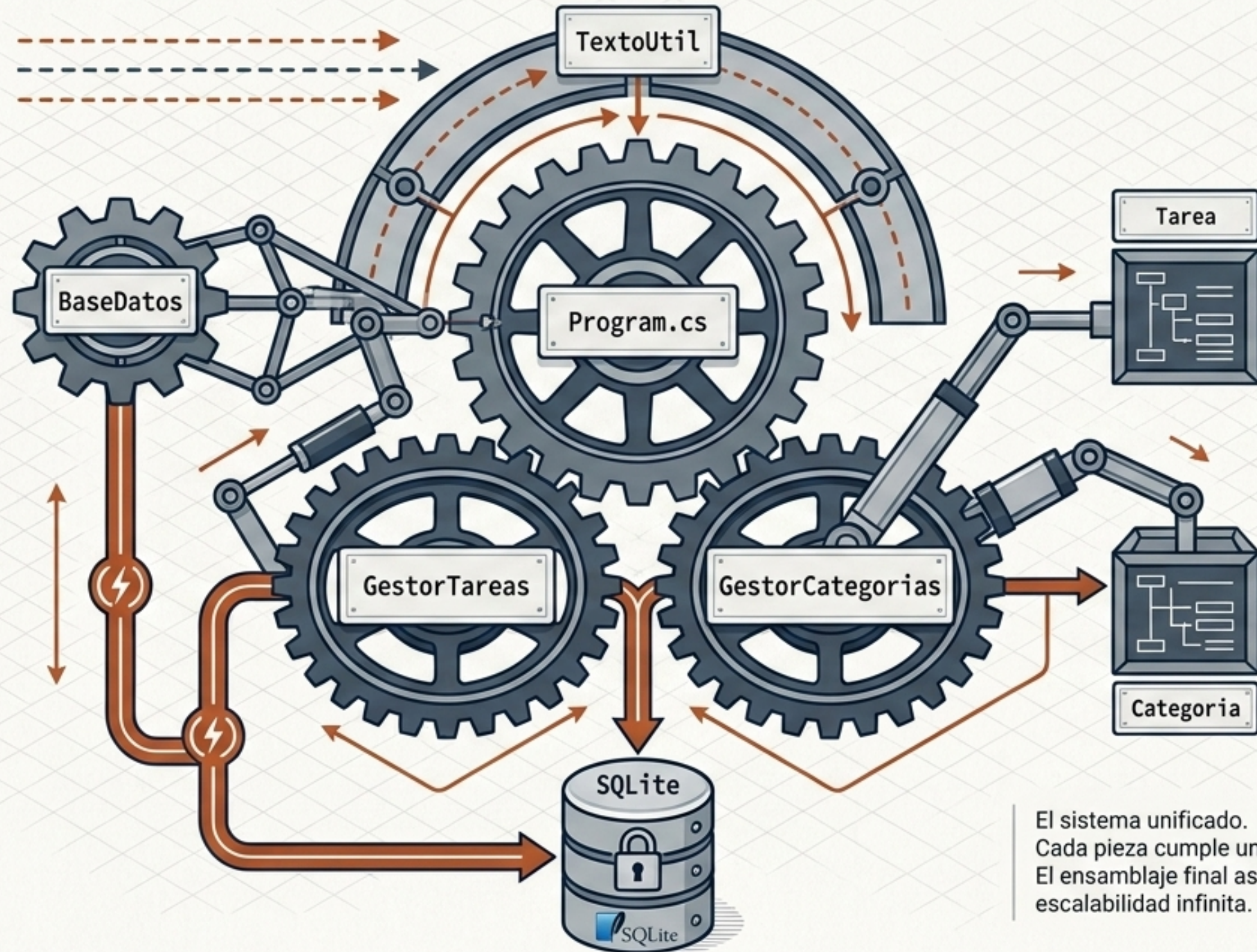
Clases estrictamente dedicadas. Los componentes como **'TextoUtil'** o **'GestorCategorias'** son lo suficientemente agnósticos para ser extraídos y reutilizados en futuros proyectos de C# sin modificaciones.



Seguridad de Recursos

La implementación sistemática de bloques **'using'** alrededor de **'SqlConnection'** previene activamente fugas de memoria y bloqueos irrecuperables (Database Locks) en el archivo físico.

Síntesis: El Ecosistema C# y SQLite



El sistema unificado.
Cada pieza cumple un único rol.
El ensamblaje final asegura
escalabilidad infinita.

Siguientes Pasos: Pon a Prueba el Motor

Una arquitectura de software no está terminada; está preparada para evolucionar.

NIVEL 1: Expansión

Añade una nueva propiedad a la Entidad Tarea (por ejemplo: `'private DateTime fechaLimite;'`).
Actualiza el Gestor para que inserte este nuevo dato en tu tabla SQLite.

NIVEL 2: Nueva Arquitectura

Diseña desde cero una nueva entidad llamada 'Usuario'.
Crea su respectivo 'GestorUsuarios' emulando la separación de responsabilidades que has aprendido.

NIVEL 3: Integración Relacional

Modifica la estructura de la base de datos y de la aplicación para que cada objeto 'Tarea' esté obligatoriamente asignado y referenciado a un objeto 'Usuario'.

El código es tuyo. Descarga el repositorio fuente y comienza a implementar estas misiones en Visual Studio hoy mismo.